

Verdant Coil – Scene & Infrastructure Architecture

This document outlines all major scenes, systems, and reusable components required to implement *Verdant Coil* in Godot. It is intended as a comprehensive map for building, organizing, and scaling the project.

Core Game Scenes

1. `Main.tscn` — *Game Root*

Purpose: Main hub for switching between modes and menus. - **Contains:** - `MainMenu` - `ExploreMode` - `BuilderMode` - Optional: `SettingsMenu`, `ProfilePanel` - **Script:** `main.gd` - **Role:** Handles transitions and global player data.

2. `ExploreMode.tscn` — *Level Exploration*

Purpose: Play through a Coil level. - **Contains:** - `TileMap` or `TileContainer` - `Crawler` (player agent) - `TurnManager` - `UI` with `ResourceBar`, `UpgradeBar`, and menu buttons - **Script:** `explore_mode.gd` - **Role:** Run per-turn logic, handle triggers, detect win condition.

3. `BuilderMode.tscn` — *Level Builder*

Purpose: Build and validate Coils. - **Contains:** - `TileGridEditor` - `BiomassManager` - UI: `TilePalettePanel`, `BiomassBar`, `Validate/Save/Publish/Undo` - **Script:** `builder_mode.gd` - **Role:** Drag-to-place tiles, validate paths, manage biomass and export.

4. `Crawler.tscn` — *Player Unit*

Purpose: Represents the user-controlled entity. - **Contains:** - `Sprite2D`, `CollisionShape2D` - `UpgradeController` - **Script:** `crawler.gd` - **Role:** Movement, upgrades, interaction with hazards and environment.

Tile System

5. `BaseTile.tscn` (abstract)

- **Variants:** `FleshTile`, `AcidPoolTile`, `DigestWallTile`, `HeartrootTile`, `PulseGateTile`, etc.
 - **Script:** `tile.gd` or per-tile subclass
 - **Role:** Defines tile type, biomass cost, trigger/hazard logic, upgrade checks.
-

Systems and Managers

6. TurnManager.gd (Autoload)

Handles: 1. Crawler move 2. AOE resolution 3. Triggers fire 4. Projectiles act 5. Enemies (future)

7. LevelValidator.tscn

- **Script:** level_validator.gd
- **Role:** Ensures Heartroot is reachable; identifies required upgrades.

8. LevelDataManager.gd

- **Role:** Load/save JSON level files; manage metadata.

9. UpgradeController.gd

- **Role:** Tracks upgrade toggles and communicates with tiles and Crawler.

10. BiomassManager.gd

- **Role:** Tracks biomass budget and tile costs in builder.

11. PlayerProfile.gd

- **Role:** Stores unlocked upgrades, earned resources, play history.

12. GameStateManager.gd (Autoload)

- **Role:** Controls global state (pause, reset, transitions, notifications).

13. ProjectileManager.gd

- **Role:** Pool and update projectiles (e.g., Guard Nodule, Spine Shooter).

UI Scenes & Reusables

14. ResourceBar.tscn

- Shows Nutrient and Sporeprint amounts.

15. UpgradeBar.tscn

- Toggle active upgrades (, , ).

16. `TilePalettePanel.tscn`

- Scrollable list of tiles with biomass cost and tooltips.

17. `ActionButtons.tscn`

- Save, Validate, Publish, Undo, Restart, Menu.

18. `Tooltip.tscn`

- Floating contextual info bubble.
-

File, Profile, and Metadata

19. `LevelBrowser.tscn`

- UI to browse local or imported levels; display metadata, play count, rating.

20. `Publisher.gd`

- Handles level publishing: move to `Published/`, generate metadata.

21. `MetadataViewer.tscn`

- Shows level title, required upgrades, author, difficulty.
-



Debug and Dev Tools

22. `DevConsole.tscn`

- In-editor cheats, unlocks, test pathfinding, teleport crawler.
-

Future/Optional Scenes (Post-MVP)

23. `OnlineLevelSync.tscn`

- Connect to backend, fetch and submit levels.

24. `AccountManager.tscn`

- UUID auth, sync progression, unlocks.

25. AnalyticsLogger.gd

- Logs play counts, tile usage, level completions (for backend use).

This architecture supports modular, scalable development and clean UI separation, and adheres to Godot's best practices for maintainable 2D games.